
FONDAMENTI DI PROGRAMMAZIONE B

Tempo a disposizione: 2 ore 15 minuti

Nome Cognome Matricola

Esercizio 1 [C++] (15pt). Definire una classe template `Queue<T>` che realizza il tipo di dato astratto coda di elementi di tipo `T` che permette di inserire un nuovo elemento in coda e di rimuovere un elemento dalla testa della coda (FIFO: First In First Out). La classe deve definire:

- ▶ un costruttore senza argomenti che crea una coda vuota
- ▶ un metodo `enqueue` che, preso come argomento `elem`, aggiunge `elem` in coda
- ▶ un metodo `dequeue` che rimuove l'elemento in testa alla coda e lo restituisce. Se la coda è vuota, il metodo deve lanciare un'eccezione.
- ▶ un metodo `isEmpty` che controlla se la coda è vuota

Si sovraccarichi l'operatore `<<` in modo tale che stampi gli elementi della coda su uno stream di output `fout` nel formato $[e_1, e_2, \dots, e_n]$. Non è consentito utilizzare classi della STL. Se necessario, ridefinire gli opportuni metodi, costruttori e/o operatori. Specificare opportunamente eventuali metodi e parametri costanti. Massimizzare incapsulamento e information hiding.

Esercizio 2 [Java] (15pt).

- ▶ Si implementi una classe astratta `Impiegato`. Ogni impiegato è caratterizzato da un nome, un cognome e le ore che l'impiegato ha lavorato in un anno (si definiscano i metodi getter per gli attributi corrispondenti). La classe `Impiegato` deve ridefinire il metodo `equals`. Due impiegati sono uguali se hanno lo stesso nome, lo stesso cognome e se hanno lavorato lo stesso numero di ore. Inoltre, deve definire un metodo astratto `getStipendioAnnuale` che ritorna un valore di tipo `double`.
- ▶ Si implementino le classi `ImpiegatoBase` e `Manager`, sottoclassi di `Impiegato`. Per l'implementazione del metodo `getStipendioAnnuale` nelle classi `ImpiegatoBase` e `Manager`: un `ImpiegatoBase` ha un importo orario pari a 15.50€, mentre un `Manager` di 25.90€.
- ▶ La classe `ImpiegatoBase` deve mettere a disposizione un unico costruttore che prende come parametri un nome, un cognome e le ore che l'impiegato base ha lavorato in un anno ed inizializza i campi corrispondenti. Si ridefinisca il metodo `equals`. Due impiegati base sono uguali per il metodo `equals` se hanno lo stesso nome, lo stesso cognome e le stesse ore lavorate.
- ▶ La classe `Manager` è caratterizzata da un team, ovvero un insieme di impiegati di base che un manager gestisce (un manager quindi non può avere nel team altri manager). La classe `Manager` deve mettere a disposizione un unico costruttore che prende come parametri un nome, un cognome e le ore che il manager ha lavorato ed inizializza i campi corrispondenti. Quando costruito, un manager non ha nessun impiegato nel team.

Si ridefinisca il metodo `equals`. Due manager sono uguali per il metodo `equals` se hanno lo stesso nome, lo stesso cognome, le stesse ore lavorate e lo stesso team.

Inoltre, la classe `Manager` deve mettere a disposizione un metodo `aggiungiAlTeam` che prende come parametro un impiegato e lo aggiunge al team. È possibile aggiungere un impiegato al team se ha lavorato almeno 20 ore nell'anno. Se l'impiegato è un manager, l'impiegato fa già parte del team del manager oppure l'impiegato ha lavorato meno di 20 ore nell'anno, il metodo deve lanciare un'eccezione di tipo controllato `TeamManagerException`, da implementare. Infine, la classe `Manager` deve essere comparabile (deve implementare l'interfaccia `Comparable<Manager>`). Il metodo `compareTo` utilizza il numero di impiegati nel team dei manager per il confronto.

- ▶ (+3pt) La classe `Manager` deve essere iterabile (deve implementare l'interfaccia `Iterable<Impiegato>`), in modo tale che sia possibile iterare sugli impiegati del team del manager.

N.B. Massimizzare incapsulamento e information hiding. Non è richiesta l'implementazione del metodo `hashCode` per le classi richieste. Per ciascuna classe, è possibile supporre di avere a disposizione una implementazione del metodo `hashCode` coerente col metodo `equals` che implementerete.